



PROJET DOOM LIKE 1993

HENRY ROYER et LORIS SOGLIUZZO

Objectifs



**Création d'un FPS en 2.5D
à la DOOM 1993**



**Gestion d'un architecture
client server**

Stack techniques utilisés



C++ (Standard 17)



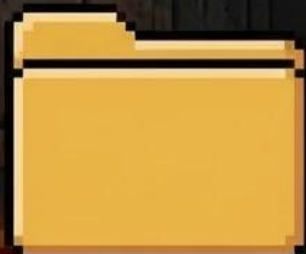
SDL2



OpenGL

Architecture Globale du Code

Structure Modulaire



src/Engine/

Classes de rendu
(CameraRendering,
Maze, Menu)



src/Network/

Gestion communication
(NetworkClient,
NetworkServer,
PacketFactory)

Modèle Client-Serveur



Serveur

Autorité centrale,
gère la seed du
labyrinthe et la
position des
entités.



Client

Affiche l'état du
monde et envoie
les inputs.

Logique du Jeu et Génération Procédurale

Classe Maze (Labyrinthe)

```
template <size_t Size>
struct Maze
{
    ...
}
```

Taille définie à la
compilation
(Optimisation mémoire).

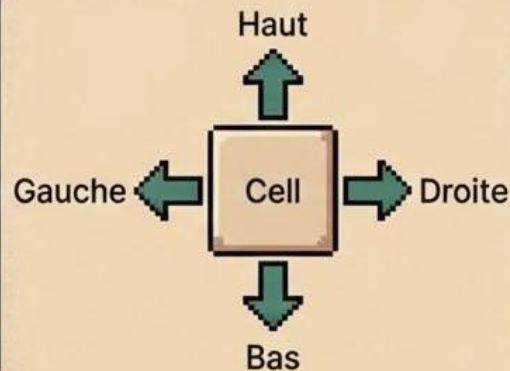


Génération Procédurale



Seed unique = Même labyrinthe
(Synchro réseau).

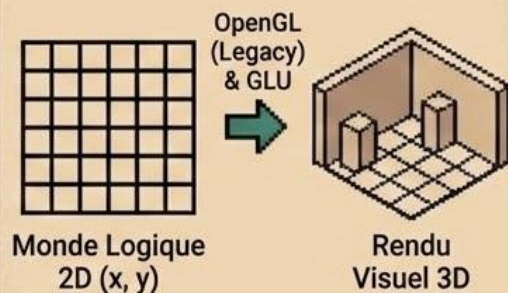
Structure de Données



Grille de Cell avec
connexions booléennes.

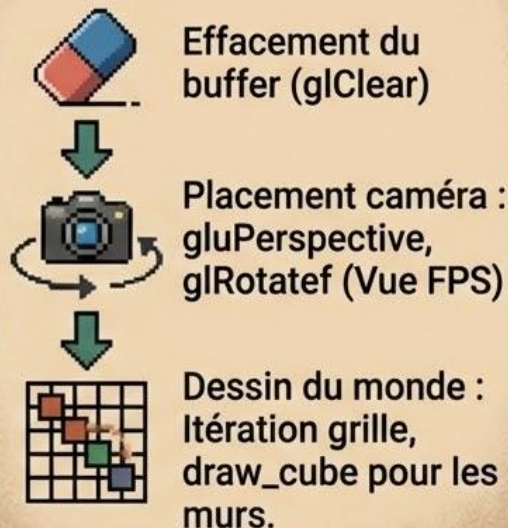
Le Moteur de Rendu (2.5D)

Technique 2.5D



Monde logique en 2D
→ Rendu visuel en 3D.

Pipeline de Rendu (CameraRendering)



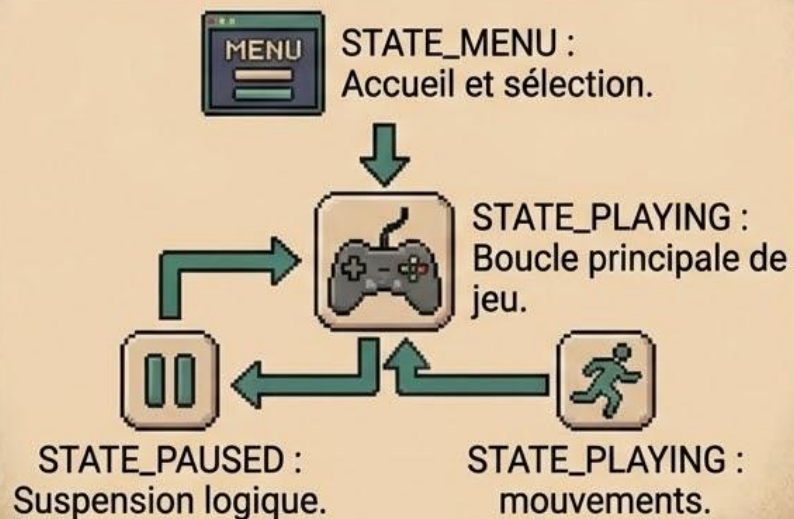
Gestion des Textures



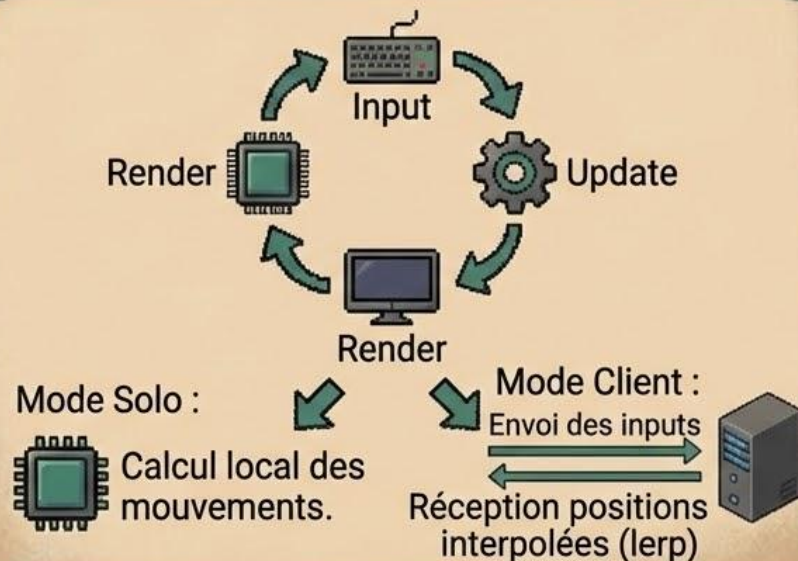
Conversion en textures
OpenGL (murs, sol, ennemis).

Boucle de Jeu et États

Machine à États (Game_State)



Boucle de Rendu (rendering_loop)

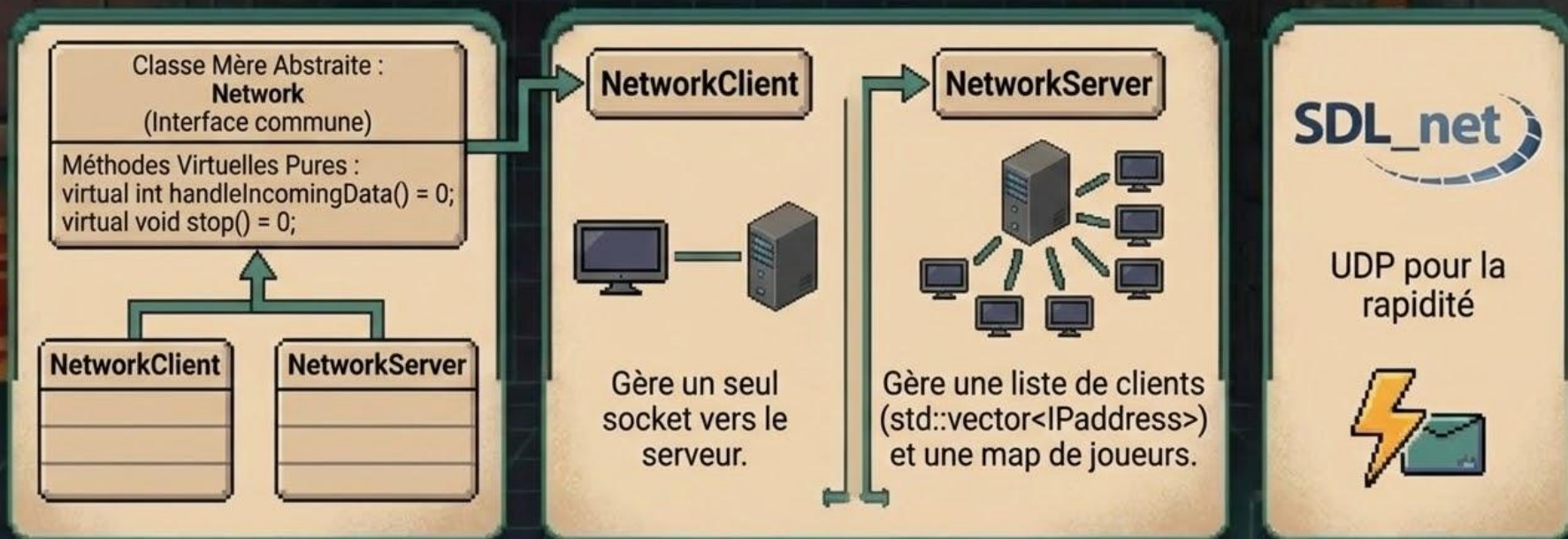


Architecture Réseau & Polymorphisme

Abstraction (Héritage)

Implémentations Spécifiques

Bibliothèque



Protocole et Design Pattern 'Factory'

Structures de Données (PODs)

```
struct Packet {  
    uint8_t type;  
    // ... données  
} __attribute__((packed));
```

Utilisation de struct avec #pragma pack pour la sérialisation binaire directe.

Types de paquets :



InputPacket
(Client->Serveur)



WorldStatePacket
(Serveur->Client)



SeedPacket

Design Pattern : Packet Factory



PacketFactory

Classe PacketFactory avec méthodes statiques.

Rôle : Centraliser la création et le remplissage des UDPpacket.



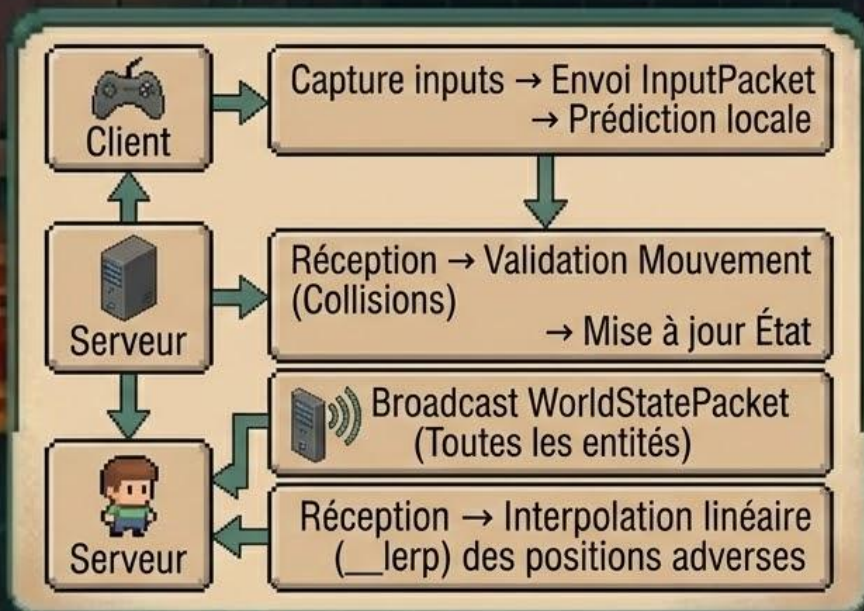
UDPpacket

Avantage : Séparation de la logique de création et d'envoi.

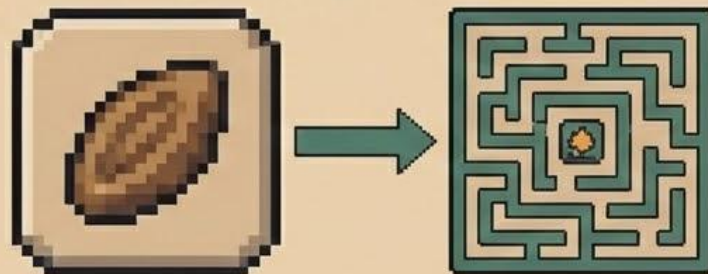


Boucle de Synchronisation et 'Authoritative Server'

Cycle de Mise à Jour



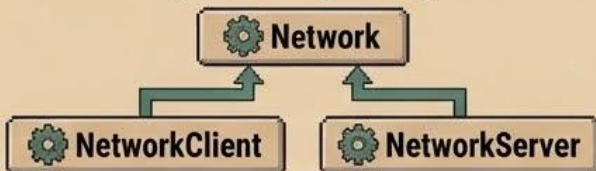
Synchronisation Initiale



Envoi de la Seed du labyrinthe
à la connexion
(Garantie de cohérence)

Synthèse des Concepts P00

Héritage & Polymorphisme



- Architecture Network → NetworkClient / NetworkServer.
- Permet de traiter le réseau de manière générique dans la boucle principale.

Encapsulation



Masquage des détails bas niveau (Sockets UDP, IPAddress, std::vector de clients) via des accesseurs et méthodes privées.



Exemple : `handleIncomingData()` gère la réception sans exposer le buffer brut.

Méthodes Statiques & Design Patterns



UDPpacket

- Pattern Factory (PacketFactory) pour la création d'objets complexes.
- Méthodes de classe pour l'organisation utilitaire sans instantiation.

Templates (Généricité)

`Maze<Size>`

Classe `Maze<Size>` pour paramétrer la taille du jeu à la compilation.



Size=Small



Size=Large

Conclusion et Perspectives

Bilan & Difficultés



- Défi Technique : Synchronisation multijoueur (Interpolation & prédiction).



- Apprentissage : Projet ambitieux nécessitant la découverte simultanée d'OpenGL (Rendu) et des Sockets (Réseau).

Perspectives d'amélioration (To-Do List)

Expérience Utilisateur (UI)



- Lancer le serveur directement depuis le menu du jeu.
- Champ de texte pour saisir l'IP du serveur (actuellement en ligne de commande).

Gameplay (Pour en faire un vrai Doom)



- Système de tir (Raycasting offensif).
- Gestion de la vie (Health) et des dégâts.
- Ajout de monstres (IA) en plus des joueurs adverses.

